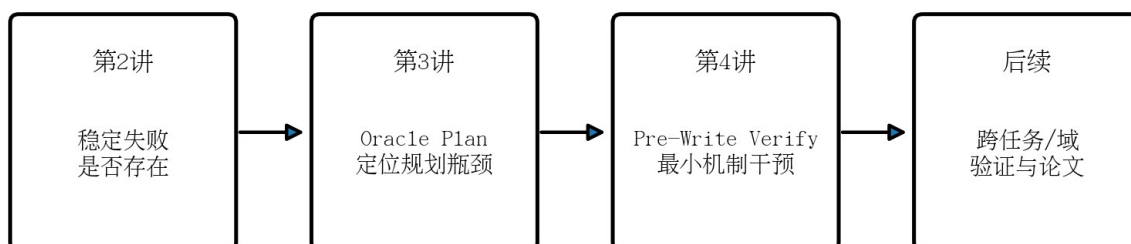


Agent Systems 研究型进阶专题 A · 第 4 讲

从 Oracle 诊断到最小机制干预：Pre-Write Verification

本讲定位：第 3 讲用 Oracle Plan 回答“规划是不是主要瓶颈”。第 4 讲第一次真正设计一个不依赖 gold answer 的小方法：只在 Agent 准备执行不可逆 WRITE tool 时触发验证。



科研升级：从“给完整答案做诊断”缩小到“只在不可逆写操作前加一道验证门”

1. 为什么不能停在 Oracle Plan ？

Oracle Plan 很适合诊断，因为它把“规划负担”大幅削弱：如果完整计划一给，失败大量消失，我们就知道规划/决策值得继续追。但是它不能作为最终方法，因为它使用了 benchmark 的 reference resolution steps，相当于考试时给了部分标准解题步骤。真实系统拿不到这些信息。

科研上的下一步不是“再换一个更强模型”，而是把 Oracle 中真正有用的部分尽量缩小：究竟是完整计划有用，还是只需要在关键决策点多检查一次？如果后一种成立，就可能从诊断工具走向可部署机制。

核心原则：最小干（minimal intervention）：只在最可能产生不可逆错误的位置增加额外机制，其余时间尽量保持与 Baseline 一样。

2. 先理解 READ 与 WRITE：为什么 WRITE 是界？

τ^3 -bench 的工具系统把工具按概念分成 READ、WRITE、THINK、GENERIC。READ 主要查询信息，不改变数据库；WRITE 创建、更新或删除状态。对 Agent Systems 来说，这个边界非常重要：读错了通常还能重新查，写错了可能已经把订单、退款、航班或账户状态改掉。

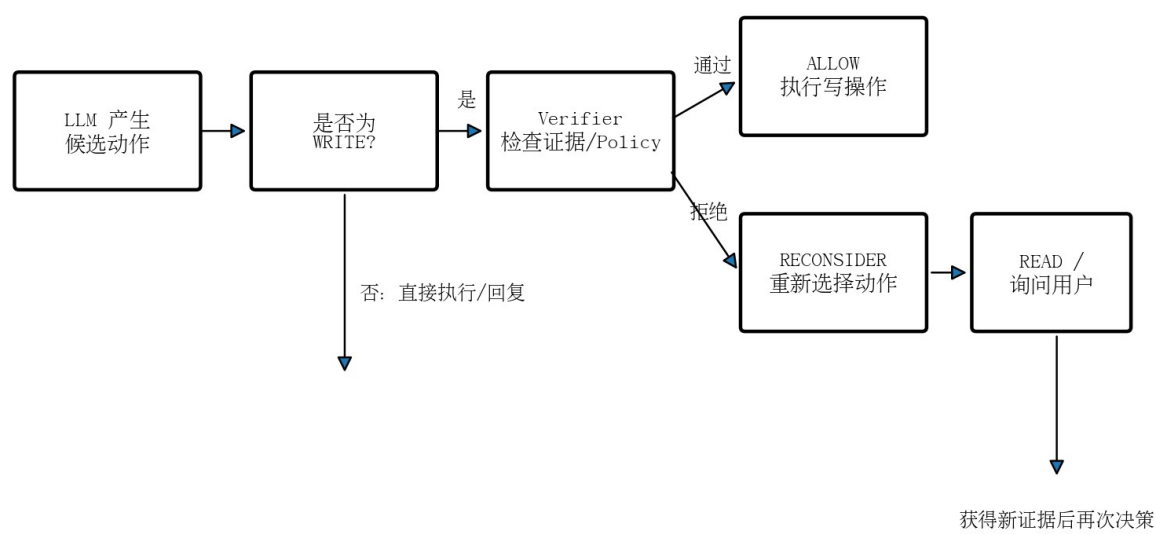
工具类型	直观含义	错误后果	本讲处理
READ	查订单、查用户、查政策所需信息	通常可恢复	不增加 verifier
WRITE	退款、修改、取消、创建	可能改变系统状态	写之前验证
THINK	部推理	不改变环境	不处理

GENERIC	通用工具	视具体作用	默认不按 WRITE 理
---------	------	-------	--------------

底层上，tau2 的 ToolKit 使用 ToolType.WRITE 标记写工具，并默认把 WRITE 视为 mutates_state=True。本讲的自定义 Agent 会读取这个元数据，而不是靠“函数名里有没有 update/refund”这种脆弱的字符串猜测。

3. Pre-Write Verification 到底做什？

Pre-Write Verification: 只在写操作前触发的最小控制环



普通 LLM Agent 每一直接生成一回或工具用。本的 PreWriteVerifyAgent 先让同一个模型正常产生“候选动作”。如果候选是 READ 或普通文字回复，完全不增加额外步骤；只有当候选包含 WRITE tool 时，才调用一个没有工具权限的 verifier。

Verifier 不知道 gold trajectory，也不允许调用工具。它只看当前 policy、对话历史和“即将执行的写操作”，判断三件事：

- Policy 是否允许这个写操作；
- 关键前置条件是否已经满足；
- 关键参数（用户、订单、金额、目标状态等）是否有当前证据支撑，而不是模型猜出来的。

如果通过，原 WRITE 正常执行；如果拒绝，原写操作不会进入真实环境，Agent 会得到一条临时内部反馈，重新选择 READ、询问用户或其他动作。

4. 和 模型多想一”有什？

区别在于触发位置和可检验性。ReAct 或全程 chain-of-thought 会在每一步增加思考；Pre-Write Verification 只对 WRITE 边界增加检查。因此我们可以提出更尖锐的预测：如果问题真的是“在证据不足时过早执行不可逆动作”，那么这类错误应该明显下降；而纯粹的阅读理解错误、通信错误、API 基础设施错误不应该被普遍修复。

“机制预测”比“涨分”更重要

Failure 类型	干预后预测	解释
过早 WRITE / 参数无依据	应明显减少	支持目标机制
READ 顺序不同但最终正确	不应明显变化	避免过度干预
纯理解/通信失败	通常不应被修复	负向对照
基础设施/API 错误	不应变化	必须单独剔除

科研价值：一个方法如果“什么失败都能改善”，反而不利于机制解释。真正有价值的机制结果往往具有选择性：只改变理论上应该改变的那类失败。

5. 本讲提出的可证伪假设

H2：对于第 3 讲中被 Oracle-plan 显著改善、且 First Bad Span 位于过早/错误 WRITE 决策附近的任务，仅在 WRITE 前加入证据与 policy 验证，就能恢复相当一部分稳定性；这种改善应主要集中在 WRITE-related failures。

H2 可能被否定。如果 PreWrite 并没有改善这些任务，或者它对一个纯通信失败任务也同样大幅改善，那么“写前验证是关键机制”这个解释就需要降级。科研上，被否定不是失败；它是在排除错误机制。

6. 为什么要保留负向对照任务？

第 3 讲教学例子中，Task 17 和 41 在 Oracle-plan 下从低成功率提升到 1.0，而 Task 23 只从 0.0 提升到 0.25。第 4 讲因此把 17/41 当作目标任务，把 23 当作负向对照。

如果 PreWrite 只在 17/41 上改善，而 23 基本不动，这与 H2 的选择性预测一致；如果所有任务都一起大涨，可能只是“多调用一次模型”带来的通用推理增强，而不一定是 WRITE 边界机制。

7. 代码层面：这个 Agent 去？

官方 Agent Developer Guide 提供了两条关键能力：第一，可以实现自己的 HalfDuplexAgent 或继承 LLMAgent；第二，可以用 registry.register_agent_factory() 在运行时注册新 Agent。我们利用这两点，不修改 benchmark 数据、不改 evaluator，也不需要永久修改 tau2 源码。

本资料包的 prewrite_verify_agent.py 继承官方 LLMAgent，并覆盖 _generate_next_message()：普通候选动作仍由官方 generate() 产生；仅当工具元数据显示为 ToolType.WRITE 时才插入 verifier。

```
candidate = generate(..., tools=self.tools)
```

```

if not contains_write(candidate):
    return candidate
verdict = verify_without_tools(candidate)
if verdict == 'ALLOW':
    return candidate
return regenerate_after_feedback(...)

```

为什么这叫 Harness-level intervention：我们没有重新训练模型；我们改变的是模型与工具环境之间的执行编排。Agent Systems 的很多科研问题正发生在这一层：什么时候允许行动、什么时候验证、什么时候回退。

8. 你需要记录的不只是最终 Reward

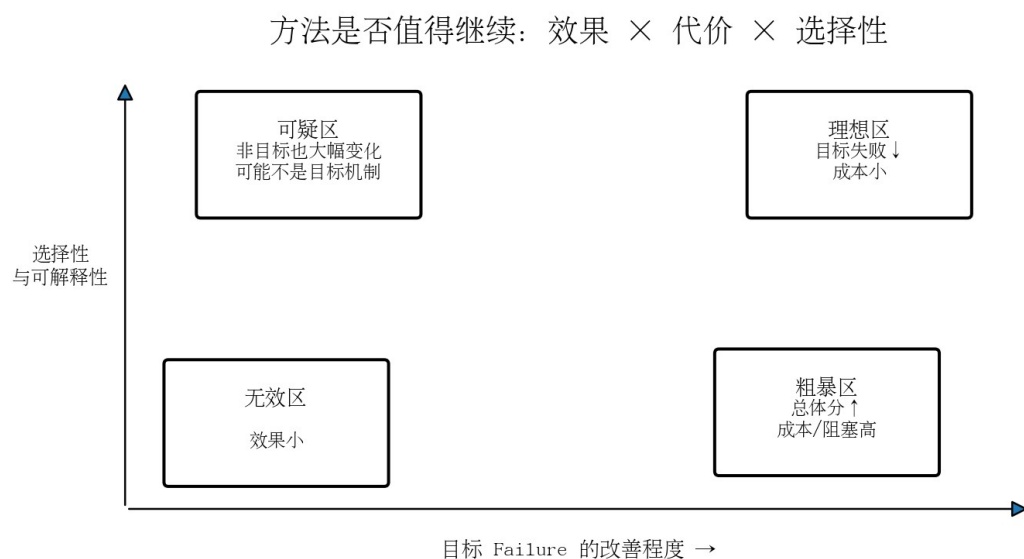
如果只记录最终 reward，我们不知道为什么变化。因此本讲把 verifier 的诊断信息写入 AssistantMessage.raw_data 中，并在结果分析脚本里统计：

- verification_triggered：一共触发多少次写前验证；
- ALLOW / RECONSIDER 次数；
- 被拒绝的 proposed WRITE tool 名称；
- reconsider 后最终动作是否变成 READ/询问/不同 WRITE；
- 每个 task 的 reward、pass¹-pass⁴、agent_cost 和 duration。

这样我们可以从“PreWrite 分数更高”进一步追问：“它究竟拦住了多少次写操作？这些拦截是否集中在原来失败的 episode？成功提升是否伴随合理的成本？”

9. 成本延什是方法的一部分？

PreWrite 的代价很明确：每次候选 WRITE 至少多一次 LLM verifier 调用；如果被拒绝，还可能多一次重新生成。因此它可能提高可靠性，也可能增加 token、美元成本和延迟。真实系统研究不能只报告成功率。



本讲采用一个最简单的“方法是否值得继续”判断：目标失败有明显下降；负向对照没有同等幅度改善；额外成本与延迟在可接受范围；没有引入大量“本来正确的写操作被阻塞”的新失败。

10. False Positive 与 False Negative：Verifier 也会犯错

Verifier 不是规则引擎，它仍然是 LLM。它有两种典型错误：

错误	含义	系统后果	Trace 中怎么发现
False Positive (误拦截)	正确 WRITE 被判 RECONSIDER	任务变慢甚至失败	Baseline 正确写入，PreWrite 却反复查/询问
False Negative (漏拦截)	错误 WRITE 被判 ALLOW	机制没有发挥作用	Verifier 日志显示 ALLOW，随后 DB/communication 失败

这一步很重要，因为你的研究对象已经从“原 Agent 会不会犯错”变成“Agent + Verifier 组成的新系统怎样犯错”。这正是 AI Systems Science 的视角。

11. ：保持些量不？

变量	Baseline	PreWrite	说明
Domain / tasks	同一 retail task IDs	完全相同	避免任务差异
Agent model	同一模型	同一模型	不换更强模型
User simulator	相同	相同	避免交互分布变化
num_trials	4	4	重复随机实验
seed	300	300	生成同一 trial seed 序列
Evaluator	官方默认	官方默认	不改评分
Harness	LLMAgent	PreWriteVerifyAgent	核心干预

实验规模继续小：3 tasks × 4 trials × 2 conditions = 24 trajectories。只有当你确认脚本、日志、机制信号都合理后，才扩大到更多 tasks。

12. 果出以后，“强度”？

- Level 0：只看到总分变化——只能说有性能差异。
- Level 1：目标任务改善——可以说与 H2 一致。
- Level 2：Trace 显示错误 WRITE 被 verifier 截且 First Bad Span 消失/后移——机制证据增强。
- Level 3：负向对照不变、跨 task 重复、成本可控——开始形成方法性证据。
- Level 4：跨 domain / model 复现并做更细 ablation——才逐渐接近论文级结论。

13. 常见误区

- 误区 1：PreWrite 涨分 = 方法成功。——还要看是不是目标失败减少，以及成本和误拦截。

- 误区 2：Verifier 用同一个模型就没有意义。——意义在于系统编排改变了决策时机和信息结构；但必须报告额外调用成本。
- 误区 3：把 verifier 的判断当真值。——它也是随机、会犯错的 AI 组件。
- 误区 4：只跑最容易被救回的两个任务。——至少保留一个不符合机制预期的负向对照。
- 误区 5：看到 2-3 个 task 就写成普遍规律。——本讲只是 pilot，后续要扩样本。

14. 本讲结束时你应掌握的“最低闭环能力”

- 从 Oracle 诊断中提炼一个不依赖 gold 信息的最小机制。
- 实现一个真正插在 Agent-Tool 边界上的 Harness intervention。
- 同时记录 outcome、process、cost、latency 四类证据。
- 用 target task + negative control 检查机制选择性。
- 区分“有效方法”“机制证据”“小样本 pilot”三个不同层次的结论。

15. 官方依据与版本说明

本讲继续按 tau2-bench v1.0.1 的 text half-duplex 架。官方 [Agent Developer Guide](#) 明确支持自定义 HalfDuplexAgent/LLMAgent，并通过 factory + registry 注；官方 [environment/toolkit.py](#) 定义 ToolType.READ、WRITE、THINK、GENERIC，并把 WRITE 默认视为 mutates_state=True。本讲只利用这些公开架构接口，不修改任务 gold、数据库或 evaluator。

参考源码位置：[src/tau2/agent/README.md](#)、[examples/agents/react_agent.py](#)、[src/tau2/agent/llm_agent.py](#)、[src/tau2/registry.py](#)、[src/tau2/environment/toolkit.py](#)。